

MANUAL26. УРОК 13. РЕГРЕССИЯ: КАК ПРОВЕРЯТЬ СТАБИЛЬНОСТЬ ПОСЛЕ ИЗМЕНЕНИЙ.

Почему один успешный retest не гарантирует безопасность релиза?

Потому что исправление дефекта почти всегда затрагивает соседние зоны. Вы починили конкретный сценарий, но могли случайно изменить поведение в других частях продукта. Поэтому после retest нужен еще один уровень защиты — регрессия.

Чем регрессия принципиально отличается от retest?

Retest отвечает на вопрос:
«исправленный дефект больше
воспроизводится?»

Регрессия отвечает на вопрос:
«не сломали ли мы рядом то, что
раньше работало?»

Без этого второго вопроса
команда видит локальный успех
и легко пропускает системный
риск.

Почему именно в релизной фазе регрессия становится критичной?

Потому что перед релизом
изменений много, времени мало,
а цена ошибки максимальна.
Любой пропущенный побочный
дефект в этот момент может
ударить по удержанию
пользователей, выручке или
репутации продукта.

Когда регрессия обязательна без компромиссов?

В первую очередь после правок
в критичных потоках:

1. авторизация и сессии;
2. платежи и магазин;
3. сохранение прогресса;
4. матчмейкинг или подбор сессий;

5. интеграции с внешними сервисами;
6. рефакторинг ядра или серия hotfix.

В этих зонах лучше проверить меньше, но точно, чем «широко и поверхностно».

Почему опасно стремление «проверить все подряд»?

Потому что полный прогон в ограниченном времени часто дает обратный эффект: команда распыляется, устает и не успевает закрыть действительно критичные риски. Регрессия эффективна, когда построена по вероятности поломки и цене последствий.

Как построить регрессию на основе риска?

Рабочий алгоритм:

1. собрать карту изменений;
2. определить ключевые пользовательские цели;
3. добавить исторически проблемные зоны;

4. выделить must-pass;
5. сформировать расширенный блок на случай доступного времени.

Так появляется прозрачный приоритет вместо случайного списка.

Что обычно входит в must-pass набор?

Туда включают сценарии, без которых продукт теряет базовую ценность для пользователя: вход, основной цикл действия, подтверждение результата и корректный выход из сценария. Если этот слой не прошел, обсуждать расширенную регрессию рано.

Почему волновой подход помогает не утонуть в объеме?

Потому что он фиксирует порядок:

1. волна 1 — критичный минимум;
2. волна 2 — риск-зоны с высокой ценой ошибки;
3. волна 3 — расширение при наличии окна.

Даже если время внезапно сжимается, у команды остается проверенный базовый слой.

Как это работает на кейсе Valorant?

Починили дефект «поиск матча не стартует». Retest показал, что поиск снова работает. Но регрессия обнаружила: при отмене поиска лобби иногда зависает, а обновление ранга задерживается. Если бы команда ограничилась retest, эти проблемы ушли бы в релиз.

Почему регрессионный набор нельзя держать неизменным?

Потому что продукт и риски меняются. Набор, который был идеальным три месяца назад, может сегодня пропускать актуальные поломки. Его нужно пересматривать после релиза, крупных инцидентов и архитектурных изменений.

Что добавлять в набор после

инцидента?

Любой реальный инцидент — источник новых проверок. Если проблема прошла в прод, похожий сценарий должен попасть в будущую регрессию. Это превращает ошибки команды в накопление качества, а не в повторяющийся круг.

Как фиксировать ограничения, если времени не хватает?

Открыто и без оправданий. В отчете по регрессии должно быть:

1. что проверили;
2. что не вошло;
3. почему не вошло;
4. какой остаточный риск остается.

Команда может принять риск только тогда, когда он прозрачен.

Какие типичные ошибки совершают начинающие QA?

Чаще всего:

1. путают retest с регрессией;
 2. не отделяют критичные проверки от «желательных»;
 3. скрывают непокрытые зоны;
 4. используют один и тот же набор под любой тип изменений;
 5. не фиксируют критерии завершения;
 6. не пересматривают набор после инцидентов.
-

Как определить критерии завершения регрессии заранее?

Полезно зафиксировать до старта:

1. что обязательно должно пройти;
2. какие дефекты блокируют релиз;
3. какие риски допустимы как `known` ;
4. какой минимальный набор артефактов нужен для решения.

Тогда к концу цикла у команды меньше споров «достаточно ли проверили».

Как оформить итог регрессии так, чтобы

он был полезен команде?

Итог должен давать действие, а не описание процесса. Хороший формат: короткий статус по `must-pass`, список найденных критичных проблем, список непокрытых зон и конкретная релизная рекомендация.

Почему регрессия — это про бизнес, а не только про тесты?

Потому что через регрессию QA защищает пользовательский опыт и доход продукта. Пропущенный побочный дефект может стоить больше, чем неделя разработки. Поэтому грамотная регрессия — это элемент управления риском на уровне релиза.

Какой главный вывод урока?

Регрессия — это системная проверка последствий изменений. Она превращает тестирование из локального «починили баг» в стратегическое

«подтвердили стабильность продукта».

Какой мостик к следующему уроку?

Следующий шаг — научиться быстро решать, стоит ли вообще запускать глубокую проверку текущей сборки. В уроке 14 разберем smoke и sanity как входной контроль тестового окна.

Как понять, какие зоны чаще дают регрессию?

Обычно это области с высокой связностью: общие сервисы, пользовательские сессии, платежные подсистемы, единые UI-компоненты и интеграции. Если правка затрагивает такие точки, вероятность побочного эффекта выше даже при небольшом изменении кода.

Почему исторические инциденты важны при планировании регрессии?

Потому что прошлые инциденты показывают реальные слабые места продукта. Если конкретная зона уже несколько раз «сыпалась» после релизов, она должна иметь повышенный приоритет в следующем цикле — даже если текущее изменение выглядит локальным.

Как балансировать глубину и скорость в ограниченном окне?

Полезно заранее разделять сценарии на слои: обязательный слой для релизного решения и расширенный для доусиления уверенности. Такой подход позволяет не тратить время на низкорисковые проверки до того, как закрыт критичный минимум.

Что делать, если в середине регрессии находите blocker?

В этот момент важно не продолжать проверять «по инерции». Нужно:

1. быстро оценить влияние блокера на оставшийся набор;
2. синхронизироваться с командой по приоритету;

3. пересобрать остаток регрессии под новый риск-контекст.

Это сохраняет управляемость цикла и не расходует время на нерелевантные проверки.

Почему важно явно фиксировать «что не проверили»?

Потому что непрозрачное непокрытие — это скрытый риск. Если команда не видит непроверенные зоны, она принимает решение на неполной картине. Прозрачное непокрытие, наоборот, позволяет осознанно выбрать: переносить релиз или выпускать с known risk.

Как обновлять регрессионный набор после релиза?

После каждого релиза полезно делать короткий пост-анализ:

1. какие проверки реально поймали риск;
2. какие оказались лишними;
3. что пропустили и почему;
4. какие новые сценарии нужно добавить.

Так набор остается живым и приносит больше пользы в следующих циклах.

Какой признак говорит, что регрессия «зрелая»?

Зрелая регрессия — это когда команда может объяснить каждую проверку через риск и пользовательскую цель. Если часть шагов выполняется только «потому что всегда делали», набор требует пересмотра.

Какой дополнительный практический эффект дает этот урок?

Вы начинаете видеть регрессию как инструмент управления неопределенностью: не просто «прогон тестов», а способ обоснованно подтвердить стабильность релиза в условиях ограниченного времени.

Как применять регрессию в команде

с очень маленьким ресурсом QA?

Если QA один или окно совсем короткое, важно заранее договориться с командой о фиксированном `must-pass`, который не меняется в последний момент. Лучше стабильно закрывать один сильный набор, чем каждый релиз собирать новый хаотичный список. При этом все пропуски должны документироваться в known risks.

Что делать, если изменения пришли «в последний час» до релиза?

В таком случае полезно использовать микрорегрессию вокруг зоны изменения:

1. retest фикса;
2. два-три смежных критичных сценария;
3. проверка сохранения состояния;
4. проверка возврата в основной пользовательский путь.

Это не заменяет полный цикл, но помогает снизить наиболее опасный риск.

Почему регрессия должна быть связана с дефектной аналитикой?

Если команда ведет статистику по типам дефектов (где чаще ломается, после каких изменений), регрессия становится умнее от релиза к релизу. Без аналитики набор остается статичным и постепенно теряет эффективность, потому что проверяет «историю проекта», а не текущие точки риска.

Как объяснять продуктовой команде ценность регрессии?

Через язык последствий: «без этого набора мы не сможем уверенно подтвердить стабильность основного пользовательского цикла». Когда QA связывает проверки с целями пользователя и релизным риском, регрессия воспринимается как часть бизнес-решения, а не как задержка.

Какие сигналы показывают, что регрессию стоит автоматизировать в первую очередь?

Обычно это сценарии, которые:

1. входят в каждый `must-pass`;
2. часто ломаются после изменений;
3. требуют много ручного времени;
4. критичны для go/no-go решения.

Даже частичная автоматизация таких проверок резко повышает стабильность релизного процесса.

Как выбирать порядок сценариев внутри одной волны?

Сначала полезно ставить сценарии с наибольшей пользовательской ценностью и кратчайшим временем выполнения. Так команда быстрее получает сигнал о состоянии релиза и успевает адаптировать план, если появляются блокеры.

Почему командам полезно вести «реестр регрессионных провалов»?

Это короткий список сценариев, которые чаще всего падали в прошлых релизах. Реестр помогает не забывать повторяющиеся риски и быстрее объяснять продуктовой команде, почему определенные проверки остаются в приоритете.

Что важно вынести в практику после урока?

Перед каждым циклом фиксируйте: что точно must-pass, что пойдет во вторую волну и какие зоны останутся непокрытыми при дефиците времени. Эта дисциплина делает регрессию управляемой даже под сильным релизным давлением.

Какой быстрый индикатор зрелости

регресса можно использовать?

Если команда за первые 20–30 минут цикла может назвать текущий релизный риск и следующий приоритетный шаг, значит набор и процесс выстроены правильно.

Глоссарий к занятию

1. **Регрессия** — проверка ранее стабильного функционала после внесенных изменений.
2. **Retest** — повторная проверка конкретного исправленного дефекта.
3. **Регрессионный набор** — приоритетный список сценариев для контроля стабильности релиза.
4. **Must-pass** — минимально обязательный набор проверок, без которого нельзя принимать релизное решение.
5. **Остаточный риск** — риск, который сохраняется после завершения доступного объема проверок.